

SUBCONSULTAS EN SQL

Teoría y Casos Prácticos de Consultas Anidadas en SonoraDB.

1. Teoría de Subconsultas: Escalares vs Anidadas

Una subconsulta es una instrucción `SELECT` anidada dentro de otra instrucción SQL (que puede ser otro `SELECT`, `INSERT`, `UPDATE` o `DELETE`). Su propósito principal es aislar lógica, calcular valores intermedios o filtrar conjuntos de datos basándose en resultados dinámicos que no conocemos de antemano.

- **Subconsultas Escalares:** Son aquellas que, por definición, devuelven **un único valor** (una sola fila y una sola columna). Se utilizan habitualmente en la cláusula `WHERE` (para comparar mediante `=`, `>`, `<`) o en la cláusula `SELECT` (para proyectar un dato calculado junto a las filas).
- **Subconsultas de Lista o Anidadas Múltiples:** Devuelven una sola columna, pero **múltiples filas**. Se combinan con operadores como `IN` o `NOT IN`. Hablamos de "anidadas a tres" cuando encapsulamos subconsultas dentro de subconsultas a una profundidad de tres niveles.

EJEMPLO 1: SUBCONSULTA ESCALAR EN WHERE

Filtro por valor dinámico

Pregunta de negocio: ¿Qué canciones del catálogo duran más que la media?

```
SELECT titulo, duracion_seg FROM dbo.canciones WHERE duracion_seg > (SELECT
AVG(duracion_seg) FROM dbo.canciones) ORDER BY duracion_seg DESC;
```

titulo	duracion_seg
Fábrica 4AM	412
Marea Alta	356
Latido	330
Forja	305
Corriente Alterna	234

Análisis paso a paso:

1. **Resolución interna:** El motor SQL ejecuta primero la consulta entre paréntesis: `SELECT AVG(duracion_seg) FROM dbo.canciones`; Esta consulta es *escalar* porque devuelve un único valor matemático: 232.
2. **Truncamiento silencioso:** Ojo, la media real es 232.2. Sin embargo, al aplicar `AVG()` sobre una columna de tipo entero (`int`), SQL Server devuelve un entero truncando los decimales automáticamente.
3. **Evaluación externa:** Ese valor "232" se inyecta en la consulta principal, quedando lógicamente como: `WHERE duracion_seg > 232`. Esto nos exige de tener que recalcular manualmente el filtro cada vez que se añaden canciones nuevas al catálogo.

EJEMPLO 2: SUBCONSULTA ESCALAR EN SELECT

Proyección de métricas

Pregunta de negocio: Para las canciones de Luna Roja (artista 1), ¿cuánto se desvía su duración de la media del catálogo?

```
SELECT titulo, duracion_seg, (SELECT AVG(duracion_seg) FROM dbo.canciones) AS
media_catalogo, duracion_seg - (SELECT AVG(duracion_seg) FROM dbo.canciones) AS
diferencia FROM dbo.canciones WHERE artista_id = 1;
```

titulo	duracion_seg	media_catalogo	diferencia
Verano en Bucle	185	232	-47
Neón	201	232	-31

Análisis paso a paso:

1. **Inyección en proyección:** Usamos la misma subconsulta escalar que en el Ejemplo 1, pero esta vez dentro del SELECT. Esto adjunta el valor "232" como una constante a cada fila resultante.
2. **Operaciones aritméticas:** Volvemos a invocar la subconsulta para restarla de la duración actual de la fila (`duracion_seg - 232`).
3. **Prevención de errores en BI:** Para obtener la media con decimales basta forzar el tipo numérico multiplicando por un flotante, por ejemplo: `AVG(duracion_seg * 1.0)` devuelve 232.200000. En ingeniería de datos, olvidar este tipo de truncamientos silenciosos es una fuente clásica de errores en informes.

EJEMPLO 3: SUBCONSULTAS DE LISTA ANIDADAS A TRES NIVELES

Operador IN multinivel

Pregunta de negocio: Marketing quiere enviar una notificación a los usuarios que han escuchado alguna canción de 'Nébula'.

```
SELECT nombre_usuario, plan_suscripcion FROM dbo.usuarios WHERE usuario_id IN (
SELECT usuario_id FROM dbo.reproducciones WHERE cancion_id IN ( SELECT cancion_id FROM
dbo.canciones WHERE artista_id = (SELECT artista_id FROM dbo.artistas WHERE nombre =
N'Nébula') ) ) ORDER BY nombre_usuario;
```

nombre_usuario	plan_suscripcion
alexbeats	Premium
lucia.m	Premium
sofi.trap	Familiar

Análisis paso a paso de la profundidad (3 niveles de anidación):

1. **Nivel 3 (El más profundo):** `SELECT artista_id FROM dbo.artistas WHERE nombre = 'Nébula'`. Esta es una consulta escalar que obtiene el ID (5).
2. **Nivel 2 (Intermedio inferior):** `SELECT cancion_id FROM dbo.canciones WHERE artista_id = 5`. Devuelve una lista de IDs de canciones que pertenecen a Nébula (108, 109, 114).
3. **Nivel 1 (Intermedio superior):** `SELECT usuario_id FROM dbo.reproducciones WHERE cancion_id IN (108, 109, 114)`. Recupera qué usuarios han tocado esas canciones.
4. **Nivel 0 (Consulta Externa principal):** Extrae los datos de contacto de esos usuarios finales. **Importante:** IN

2. Tablas Derivadas (FROM)

Cuando encapsulamos una subconsulta en la cláusula FROM, el motor relacional la trata como si fuera una tabla temporal en memoria, a la cual llamamos *Tabla Derivada*. Es obligatorio asignarle siempre un alias (ej: AS t).

EJEMPLO 4: TABLA DERIVADA EN FROM

Pre-agrupación antes del cruce

Pregunta de negocio: ¿Qué usuarios han reproducido 5 o más canciones?

```
SELECT u.nombre_usuario, u.plan_suscripcion, t.num_reproducciones FROM ( SELECT
usuario_id, COUNT(*) AS num_reproducciones FROM dbo.reproducciones WHERE tipo_contenido
= N'Canción' GROUP BY usuario_id ) AS t JOIN dbo.usuarios AS u ON u.usuario_id =
t.usuario_id WHERE t.num_reproducciones >= 5 ORDER BY t.num_reproducciones DESC,
u.nombre_usuario;
```

nombre_usuario	plan_suscripcion	num_reproducciones
alexbeats	Premium	6
juanpi	Premium	5
sofi.trap	Familiar	5

Análisis paso a paso:

1. **Aislamiento de la lógica de agrupación:** La subconsulta interna calcula cuántas canciones reales ha reproducido cada usuario, excluyendo anuncios. Al empaquetar esto en (...) AS t, creamos un bloque cerrado e independiente con dos columnas: usuario_id y num_reproducciones.
2. **Cruce con el exterior:** La consulta externa trata a la tabla derivada "t" como si fuera una tabla física más de la base de datos, uniéndola con la tabla de usuarios mediante JOIN.
3. **Filtrado post-cálculo:** Una vez tenemos el resultado combinado, aplicamos el filtro requerido WHERE t.num_reproducciones >= 5.

Las tablas derivadas son una excelente alternativa a tener que crear y borrar vistas temporales, manteniendo la consulta autocontenida y limpia para lecturas complejas de Business Intelligence.

3. Subconsultas Correlacionadas

A diferencia de las subconsultas normales que se ejecutan *una sola vez* de forma independiente, una subconsulta correlacionada contiene una referencia a una columna de la consulta externa. Por tanto, conceptualmente, la subconsulta **se evalúa repetidamente, una vez por cada fila** que procesa la consulta principal.

EJEMPLO 5: SUBCONSULTA CORRELACIONADA

Búsqueda del último evento por entidad

Pregunta de negocio: Para cada usuario, ¿cuál fue exactamente su última reproducción?

```
SELECT u.nombre_usuario, r.fecha_hora, c.titulo FROM dbo.reproducciones AS r JOIN
dbo.usuarios AS u ON u.usuario_id = r.usuario_id LEFT JOIN dbo.canciones AS c ON
c.cancion_id = r.cancion_id WHERE r.fecha_hora = ( SELECT MAX(r2.fecha_hora) FROM
dbo.reproducciones AS r2 WHERE r2.usuario_id = r.usuario_id ) ORDER BY r.fecha_hora;
```

nombre_usuario	fecha_hora	titulo
sofi.trap	2026-09-18 20:03:00	Perreo Lunar
dani_dj	2026-09-19 02:15:00	Fábrica 4AM
lucia.m	2026-09-19 11:00:00	Diamantes Rotos
nachox	2026-09-19 13:40:00	Perreo Lunar
alexbeats	2026-09-20 09:05:00	Perreo Lunar
juanpi	2026-09-20 10:30:00	Rimas de Barrio
marta_rock	2026-09-20 18:01:00	Corriente Alterna

Análisis paso a paso:

- Vinculación correlacionada:** El secreto de esta consulta está en la cláusula `WHERE r2.usuario_id = r.usuario_id` de la subconsulta interna.
- Ejecución conceptual:** El motor toma la fila 1 de la tabla externa (supongamos que pertenece a 'alexbeats'). Introduce su ID de usuario en la subconsulta. La subconsulta calcula la fecha máxima *solamente* para 'alexbeats'.
- Filtro final:** Si la fecha_hora de la fila actual de 'alexbeats' coincide con la fecha máxima calculada, la fila se mantiene; si es un evento antiguo, se descarta.
- Nota de Optimización:** El usuario 'vale_indie' no aparece porque carece de reproducciones. Aunque conceptualmente la subconsulta se evalúa una vez por fila, el optimizador de SQL Server suele reescribir esto de forma eficiente bajo el capó. No obstante, en tablas con millones de registros, conviene revisar el plan de ejecución y valorar alternativas como `ROW_NUMBER()`.

4. EXISTS vs IN: Semántica y Nulos

El operador EXISTS sirve para verificar si una subconsulta devuelve al menos una fila. Se diferencia de IN en que EXISTS produce un cortocircuito lógico (se detiene en cuanto encuentra el primer acierto) y es inherentemente inmune a los problemas de los valores NULL.

EJEMPLO 6: PREDICADO EXISTS

Comprobación de existencia de registros (Cortocircuito)

Pregunta de negocio: ¿Qué artistas han publicado al menos una canción en 2026?

```
SELECT a.nombre, a.pais FROM dbo.artistas AS a WHERE EXISTS ( SELECT 1 FROM
dbo.canciones AS c WHERE c.artista_id = a.artista_id AND c.fecha_lanzamiento >= '2026-
01-01' ) ORDER BY a.nombre;
```

nombre	pais
DJ Coral	CO
Los Voltios	AR
Luna Roja	ES
MC Brisa	PR
Nébula	MX

Análisis: EXISTS actúa como un booleano. No le importa *qué* devuelve la subconsulta (por eso ponemos SELECT 1 como convención), ni le importa *cuántas* canciones haya; detiene la búsqueda del artista actual al hallar la primera coincidencia del año 2026.

EJEMPLO 7: LA TRAMPA DE NOT IN CON NULLS

Por qué preferir NOT EXISTS

Pregunta de negocio: ¿Qué canciones del catálogo no se han reproducido nunca?

```
-- Paso 1: Intento fallido que devuelve 0 filas silenciosamente SELECT
cancion_id, titulo FROM dbo.canciones WHERE cancion_id NOT IN (SELECT cancion_id FROM
dbo.reproducciones); -- Paso 3: Versión correcta con NOT EXISTS SELECT c.cancion_id,
c.titulo FROM dbo.canciones AS c WHERE NOT EXISTS ( SELECT 1 FROM dbo.reproducciones AS
r WHERE r.cancion_id = c.cancion_id );
```

cancion_id	titulo
114	Sin Cobertura
115	Latido

Análisis del fallo en NOT IN (Paso 2):

La tabla reproducciones contiene anuncios, cuyo cancion_id es NULL. La condición NOT IN (101, 103, NULL) se expande matemáticamente como: cancion_id <> 101 AND cancion_id <> 103 AND cancion_id <> NULL.

En SQL, cualquier comparación contra un NULL devuelve un estado lógico llamado UNKNOWN. Como la condición global nunca llega a ser explícitamente TRUE, ninguna fila pasa el filtro y devuelve un conjunto vacío.

5. Patrones Avanzados de Ingeniería de Datos

EJEMPLO 8: NOT EXISTS COMO ANTI-JOIN PARA CARGAS

Idempotencia en Data Pipelines

Contexto: El lote de datos del 21/09 se ha volcado en la tabla `stg_reproducciones`. En sistemas distribuidos, la app reenvía a veces filas que ya se habían cargado. El objetivo es mover solo los registros nuevos hacia la tabla final, asegurando que el script pueda ejecutarse mil veces sin generar duplicados (Idempotencia).

```
-- Paso 1. Identificar las filas que están en el Staging pero NO en el Destino
SELECT s.reproduccion_id, s.usuario_id, s.cancion_id, s.fecha_hora FROM
dbo.stg_reproducciones AS s WHERE NOT EXISTS ( SELECT 1 FROM dbo.reproducciones AS r
WHERE r.reproduccion_id = s.reproduccion_id ) ORDER BY s.reproduccion_id;
```

reproduccion_id	usuario_id	cancion_id	fecha_hora
34	6	113	2026-09-21 08:00:00
35	4	114	2026-09-21 18:30:00
36	3	110	2026-09-21 21:00:00
37	5	NULL	2026-09-21 22:00:00
38	5	104	2026-09-21 22:01:00

Paso 2. Convertir la consulta en un INSERT real:

Tomamos exactamente el SELECT anterior y lo acoplamos bajo una sentencia de inserción.

```
INSERT INTO dbo.reproducciones (reproduccion_id, usuario_id, cancion_id,
fecha_hora, segundos_escuchados, dispositivo, tipo_contenido) SELECT s.reproduccion_id,
s.usuario_id, s.cancion_id, s.fecha_hora, s.segundos_escuchados, s.dispositivo,
s.tipo_contenido FROM dbo.stg_reproducciones AS s WHERE NOT EXISTS ( SELECT 1 FROM
dbo.reproducciones AS r WHERE r.reproduccion_id = s.reproduccion_id );
```

Evaluación de Resultados y Patrón (Paso 3):

1. **Primera ejecución:** La pestaña Messages de SSMS mostrará (5 rows affected). La tabla productiva absorbe únicamente esos 5 registros que no existían.
2. **Re-ejecución accidental:** Si ejecutamos este código otra vez, el NOT EXISTS identificará que ahora todos los IDs de la tabla *Staging* ya existen en destino. La consulta no emitirá ninguna fila, y veremos (0 rows affected).
3. **Conclusión de Arquitectura:** Esta estructura evita tener que borrar la tabla antes de cargar, y es el pilar de los procesos de *Carga Incremental* que sustentan los flujos de datos modernos.

A partir de aquí, todos los modelos de negocio y analítica asumen que esta carga se ha estabilizado con 38 reproducciones válidas.